

Lists

Revise 360 lesson plan — 60 minutes

Unit	Beyond OCR J277: a Python pathway supporting 2.1, 2.2 and 2.3
Lesson	Lists (lesson 7 of the unit)
Lesson ID	pr-107
Experience ID	pr-107
Plan ID	PLAN-PR-L07
Content version	38992d62e275
Website	https://revise360.co.uk/experience.html?id=pr-107

Big question, outcomes and success criteria

Big question — Why is the last item of a list always at index `len(list) - 1`?

Prerequisites

The lesson before this one is pr-106. Pupils should already have finished it: the starter below retrieves from it.

Earlier in the unit: pr-101, pr-102, pr-103, pr-104, pr-105.

Assessable outcomes

- pr-107-o1** — know how to create a list and read one item out of it by its index
- pr-107-o2** — understand how to traverse a list in order to total, count or compare its items
- pr-107-o3** — know how to add an item onto a list and how to take one out again

Success criteria the pupil can check against

Taken from the briefs the independent activities themselves show, so these are the words the pupil sees. Their work has met the outcome when it does these things.

- The user types in three lines, one whole number on each. pr-107-s1-a5
- Store them as whole numbers, not as the text that comes back. pr-107-s1-a5
- Keep them in the order they were typed. pr-107-s1-a5
- Displaying the list shows its square brackets and commas, so `[4, 8, 1]` is right. pr-107-s1-a5
- Do not change the three numbers. pr-107-s1-a6
- Displaying the list shows its square brackets and commas, so `[3, 7, 2]` is right. pr-107-s1-a6
- The user types in four lines, one word on each. pr-107-s1-a7
- The user types in five lines: four words, then a position. pr-107-s2-a4

Vocabulary, resources and preparation

Key vocabulary — Array, Index, Traverse, `append()` and `pop()`

Resources

SLIDES	presentations/PR_L07_Lists_Lesson.pptx
WORKSHEET	worksheets/PR_L07_Lists_Worksheet.docx
ONLINE EXPERIENCE	experience.html?id=pr-107

Before the lesson

1. Sign in on the **teacher dashboard** (`teacher.html`) and open **Class logins**. Enter the class or group and the usernames, one per line, then print the login cards. A pupil cannot sign in without one, so do this before the lesson rather than in it.
2. Print the worksheet, one per pupil.
3. Open the slides and the experience once yourself, on the machine you will be teaching from, so the Python runtime and the 360 image are already cached.
4. Check headsets or devices are charged and on the school network.

How pupils get in

1. Go to `topics.html` on `revise360.co.uk`.
2. Sign in with the **username and PIN from the login card**. The class is already set against the card.
3. Choose the topic, then this lesson's card.

Worth knowing before you promise anything. In this build `APP_CONFIG.backendUrl` is empty, so logins, login cards and progress are stored on the device the pupil used. The teacher dashboard says so too. A pupil who moves to a different machine starts again, so keep a class on the same devices within a unit, and treat the paper worksheet as the record that travels.

Teaching sequence

Sixty minutes including transitions. The online block is the one to move if your lesson is shorter: see the adaptation at the end.

MIN	PHASE	TEACHER	PUPILS	EVIDENCE TO CHECK
0-5	Retrieve	Starter on the board or the worksheet, from the last lesson and this lesson's prerequisites.	Answer the starter on the worksheet.	The starter box on <code>PR_L07_Lists_Worksheet.docx</code> is filled in.
5-8	Review the starter	Take answers; correct the one most got wrong before moving on.	Correct their own starter.	A correction written, not just a tick.
8-16	Explain and model	The new idea, then the worked example, typed live rather than shown finished.	Watch the example being built, and say what each line does.	They can name the line that does the job, not just copy it.
16-19	Set up and settle	Sign-in, launch, and what to record on paper before anyone types.	Sign in, open the lesson, read the task before typing.	Everyone is at station 1 activity 1, not typing yet.
19-48	Online: work the stations in order	Pupils work; you circulate and read the editor over shoulders.	Work the stations in order, running and checking as they go.	The station badges fill in; the editor shows their own program, not the starter.
48-53	Check understanding	The hinge question, hands down, then the reteach if the class splits.	Answer the hinge question, hands down.	The split across the options, before any reteach.
53-58	Apply or extend	The challenge activity, or the exam practice on the worksheet.	The Challenge activity, or the exam practice.	worksheet, Exam practice a) to c)
58-60	Exit and set up the next lesson	The exit question and what to finish before next time.	Write the exit answer.	One sentence that answers the big question.

The explanation and the worked example

This is what the walls of the experience say, station by station. Teach the first two or three from the front before anyone puts a headset on; the rest the pupils read for themselves.

1. Making a list

- Square brackets make a list, with the items separated by commas.
- A list can start out empty and be filled by a loop as the values arrive.
- Common mistake: creating the empty list inside the loop, which throws each item away again.

Challenge on this wall: Which line must go above the loop that fills a list, and why?

2. Reading an item

- A list is indexed from 0, so the third item sits at position 2.
- `len()` gives how many items there are, which makes the last index `len() - 1`.
- Common mistake: an index past the end, which stops the program with an error.

Challenge on this wall: Which index holds the last of seven values?

3. Going through a list

- `for item in list:` hands you each value in turn.
- `for i in range(len(list)):` gives the positions instead, for when you need the index too.
- Common mistake: printing the index when the task asked for the value, or the other way round.

Challenge on this wall: When do you need the position rather than the value itself?

4. Adding and removing

- `append()` adds one item onto the end of the list.
- `remove()` takes out the first item with that value, and `pop(i)` takes out the one at position `i`.
- Common mistake: removing items while looping over the same list, which makes the loop skip some.

Challenge on this wall: What is the difference between `remove("red")` and `pop(0)`?

5. Totals and averages

- Total the list in a loop, then divide by `len()` to get the average.
- Track the largest and smallest by comparing each item against the best found so far.
- Common mistake: starting the largest at 0, which fails on a list of negative numbers.

Challenge on this wall: What should the largest-so-far start as, if the values could be negative?

6. Putting it together

- One pass can build the total, count the items and remember where the largest one was.
- Report the figures after the loop has finished, not while it is still running.
- Common mistake: finding the largest value but forgetting to record the position it was found at.

Challenge on this wall: How would you report both the largest value and where in the list it sits?

The example to model

A list holds many values under one name. Write the values inside square brackets with a comma between each one. Printing the list shows the whole of it.

```
pets = ["cat", "dog", "ant"]
print(pets)
print(len(pets))
```

It displays `['cat', 'dog', 'ant'] / 3`

Decisions to model out loud, not just the finished answer: say why you chose that name, why that line comes before the next one, and what you would try first if it did not work. The example deliberately uses different data from the task, so a pupil cannot copy it and be finished.

Retrieval and the check for understanding

Starter — The list words holds red, blue, green and teal, in that order. Write down what words[0], words[3] and len(words) each give, and the index of the last item.

Answer space: 4 lines on the worksheet.

The answer is in the teacher answers for this unit, not on this plan: a plan left on a desk should not give it away.

Hinge question

Use this one, hands down, before anyone moves on to independent work. It is a real activity from the experience (pr-107-s4-c2), so the wording a pupil sees matches the wording you say.

What will this program display?

Correct: ['cat', 'dog', 'ant'] 3

What a pupil might choose instead

- ['ant', 'cat', 'dog'] 3
- ['cat', 'dog'] 2

Count the hands on each. If more than a few of the class are on one wrong option, that option is the misconception to teach against before anyone goes on: go back to the station this question sits on and work through its example again, aloud, before the independent work.

The course's own response to a wrong answer here: The new item goes on the end and not the front, and the list is one longer than it was.

One response, not four. one response covers every wrong option; it does not say what to revisit for each one, so decide your own reteach for each option before the lesson - the list above is the start of it.

Misconceptions this lesson addresses

- creating the empty list inside the loop, which throws each item away again. [wall panel](#)
- What has to go between one value of a list and the next? [pr-107-s1-a6, a Fix it activity](#)
- an index past the end, which stops the program with an error. [wall panel](#)
- Three items, and the positions are 0, 1 and 2. Which of those does len() give? [pr-107-s2-a6, a Fix it activity](#)
- printing the index when the task asked for the value, or the other way round. [wall panel](#)
- Python has to be told which lines belong to the loop. [pr-107-s3-a6, a Fix it activity](#)
- removing items while looping over the same list, which makes the loop skip some. [wall panel](#)
- The word None is what the program stored, so something handed nothing back. [pr-107-s4-a6, a Fix it activity](#)

Guided to independent practice

STAGE	ACTIVITIES	WHAT IT IS
Guided	20	Support is in the activity: an example to run, an output to predict, one thing to change, a gap to fill.
Independent	18	The pupil writes or fixes it with the brief and nothing else.
Challenge	6	The hardest activity on each station. It is not optional and it is not extra: it is the last step of the station.

Scaffolds already in the product, and how they fade

- **Hint** is a ladder, not a door: the first rung names the idea, the next shows the shape of the code, the next how it starts, the last animates the technique on different data. A pupil asks for each rung. None of them is the answer.
- **Run** is free and unmarked; **Check my answer** marks. A pupil may run as often as they like, which is what makes trial and error safe.
- **I need help** says to ask you, and says plainly that the question still has to be completed. It does not notify anyone and it does not unlock anything.
- The worked example appears on every early activity and on none of the late ones. That fade is deliberate; it is not a missing example.

Accessibility

- **Read aloud** on every task window reads the task and its brief, with the prescribed values read as values rather than as punctuation.
- Text is 16px or larger throughout, and a question window scrolls rather than shrinking its type.
- Prescribed data is boxed and in a monospace face as well as coloured, so it still reads as code in black and white, in a high-contrast mode, and to a pupil who does not separate green from amber.
- **Lower literacy and EAL:** pre-teach the vocabulary above; let the pupil say the key fact before writing it; accept the key fact in two or three words.
- **SEND:** the sequence is compulsory but the pace is not. A pupil may finish a station next lesson; the progress saves after every answer and reopens at their earliest unfinished question.
- A headset is never required. Every experience works on a laptop or a tablet with a mouse or a finger.

Delivering the online experience

Before anyone enters

- Worksheet out, pen in hand. The rule is write first, then answer at the badge: the first answer is the one that counts.
- Say where the stations are. They are numbered around the room: 1 and 2 to the right, 3 and 4 behind, 5 and 6 to the left, and the final challenge on the floor.
- Say that a badge marked **Not yet** is not a fault. It opens when the station they are on is finished.

The stations, and what to record at each

#	STATION	ACTIVITIES	RECORD ON PAPER
1	Making a list	7	The exam calls these arrays, but a Python list is different in one way: it can...
2	Reading an item	7	Counting from 0 means an index is the number of items you have to step over to...
3	Going through a list	7	Going through every item of a list in turn is called...
4	Adding and removing	7	pop() does two jobs at once: it takes the item out and it also...
5	Totals and averages	8	Dividing by len() on an empty list crashes, so a robust program first checks the list is not...
6	Putting it together	7	Keeping the position of the largest value as well as the value itself answers which one as well as...
final	Final challenge: what does it do?	1	the answer to the station's challenge

Where to pause

Stop the class after station 2 and take answers to its challenge out loud. That is the point where a pupil who has misread the technique can still be put right cheaply.

Afterwards

Headsets off and down. Ask three pupils for the key fact from a station of your choosing, in their own words and not read off the sheet. Then the exit question.

Python: the order is not negotiable. Each activity teaches what the next one needs, so a pupil works through them in order and completes every one. A shorter lesson, homework or a slow worker does not authorise skipping: the pupil stops where they are and resumes at their earliest unfinished question next time, because progress saves after every answer.

A paper activity, a help request or a tick on this plan does not mark an online question complete, and there is no teacher bypass. If a pupil is stuck after the hint, that is the moment to teach them, not to move them on.

Assessment, evidence and unfinished work

ID	QUESTION	MARKS	WHERE
pr-107-x1	A loop over a list of six numbers begins <code>for i in range(1, 6)</code> . State which item is never looked at, and rewrite the line so that every item is included.	2	worksheet, Exam practice a); deck, Exam practice slide
pr-107-x2	Write a program that reads seven whole numbers into a list, then prints the largest and the position it is stored at, without using <code>max()</code> .	5	worksheet, Exam practice b); deck, Exam practice slide
pr-107-x3	Write a program that reads five words into a list, removes a word the user types in, then prints the words that are left and how many there are.	4	worksheet, Exam practice c); deck, Exam practice slide

Expected answers

Model answers and marking guidance are teacher-only and are not printed on this plan. They are in the unit's teacher guide pack, and the per-activity marking for this lesson is in `answers/records/pr-107.json`, which is not published with the website.

Every programming activity in this lesson is marked by running the pupil's program against its test cases, so the mark is the program's behaviour and not its appearance. The model solutions are in `answers/codebank/pr-107.json`.

Common errors

The wrong options the activities offer are the errors this lesson expects. The commonest to watch for:

- **What will this program display?** — 3, 7, 2 red, blue; [3, 7, 2] [red, blue]
- **What will this program display?** — mon wed ['mon', 'tue', 'wed', 'thu']; tue thu mon tue wed thu
- **What will this program display?** — mon done tue done wed done; mon tue wed done
- **What will this program display?** — ['ant', 'cat', 'dog'] 3; ['cat', 'dog'] 2
- **What will this program display?** — 8; 4 1 3

Unfinished work

Record where each pupil stopped, not a tick. They resume at their earliest unfinished question, and a station left half-done is the thing to open next lesson. Do not mark the lesson complete for a pupil who did not finish it: the score screen already shows what was and was not done, and that is the honest record.

Exit, next lesson and homework

Exit check

One of these, written, not a show of hands:

- Creating a list and reading an item by its index
- Traversing a list to total or compare its items
- Adding items to a list and removing them

The confidence columns on the worksheet are a supplement to this, not a substitute: a pupil who rates themselves confident and cannot answer the exit question has told you nothing.

Retrieval link to next lesson

Next is pr-108, Two-dimensional lists. Use this lesson's big question as next lesson's starter, so the first thing they do is retrieve it.

Homework

Finish any station left open, on the website, from the question they stopped at. If the class has no devices at home, set the exam practice on the worksheet instead and mark it next lesson.

Shorter lessons, longer lessons and no internet

If this lesson takes more than an hour

It will. There are 44 activities across 6 stations, and a beginner does not finish 44 of them in twenty-nine minutes. Plan it as two or three sessions of an hour: stations 1 and 2 in the first, 3 and 4 in the second, 5, 6 and the final challenge in the

third. Each session keeps the same starter-explain-model opening; the online block resumes where the class got to. Honest total: two to three hours.

A 40 to 45 minute lesson

Keep the starter, the explanation and the model at full length: cutting those is what makes the online work fail. Take the time out of the online block and the apply phase. Pupils stop where they are; their progress saves after every answer and reopens at their earliest unfinished question, so the next lesson starts there.

A shorter lesson still does not authorise skipping a question. The save-and-resume point is wherever the pupil got to; there is no way to mark a question complete without completing it, and no teacher bypass.

If the internet or the devices are unavailable

Teach from the slides and the worksheet. The slides carry the same teaching as the station walls, and the worksheet carries the key facts, the challenges and the exam practice. Set the online stations when the devices are back.

There is no offline version of the experience, and this plan does not contain one. Do not hand out a printed walkthrough of the stations as a substitute: the experiences are used on the website.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.